

VMETRIX

Technical Challenge

Query Manager — Proof of Concept

Position	Senior Java Developer
Estimated effort	6–8 hours
Deadline	3 calendar days from receipt
Tech stack	Java 8+ / Spring Boot 2.7+ / H2 (Oracle compat.)
Deliverable	Git repository (GitHub or GitLab)

NOTE: This challenge evaluates software design, code quality, and AI-assisted development practices. Financial domain knowledge is **not** required.

1. Context and objective

VMetrix is a SaaS investment management platform. One of its core modules is the **Query Manager**: an engine that allows users to build dynamic SQL queries through an API without writing SQL directly. Queries are constructed by selecting output columns and applying filters over a data model whose structure is defined in metadata.

Your goal is to build a **Proof of Concept** of this engine. Design and implement a REST API that receives a high-level query specification (columns + filters) and generates the corresponding SQL, automatically resolving JOINS between tables based on relationship metadata.

IMPORTANT: The metadata-driven design is central: table definitions, columns, data types, and relationships must **not** be hardcoded in Java. They must come from configuration (database tables, JSON/YAML files, or similar) that the engine reads at runtime.

2. Data model

The base model consists of three tables with the following relationships:

- **TRANSACTION** → **INSTRUMENT**: via INSTRUMENT_ID
- **TRANSACTION** → **PARTY** (counterparty): via COUNTERPARTY_ID → PARTY_ID
- **INSTRUMENT** → **PARTY** (issuer): via ISSUER_ID → PARTY_ID

2.1 TRANSACTION

Column	Type	Constraint	Description
TXN_ID	NUMBER	PK	Unique transaction identifier
TXN_DATE	DATE	NOT NULL	Transaction date
TXN_TYPE	VARCHAR2(20)	NOT NULL	BUY, SELL, MATURITY, COUPON, DIVIDEND
QUANTITY	NUMBER(18,6)		Units or nominal amount
PRICE	NUMBER(18,8)		Unit price
AMOUNT	NUMBER(18,2)	NOT NULL	Total transaction amount
CURRENCY	VARCHAR2(3)	NOT NULL	ISO currency code
STATUS	VARCHAR2(20)	NOT NULL	PENDING, APPROVED, SETTLED, CANCELLED
SETTLEMENT_DATE	DATE		Settlement date
PORTFOLIO_ID	NUMBER	NOT NULL	Portfolio reference
INSTRUMENT_ID	NUMBER	FK	Reference to INSTRUMENT

Column	Type	Constraint	Description
COUNTERPARTY_ID	NUMBER	FK	Reference to PARTY
CREATED_AT	TIMESTAMP	NOT NULL	Record creation timestamp

2.2 PARTY

Column	Type	Constraint	Description
PARTY_ID	NUMBER	PK	Unique entity identifier
PARTY_NAME	VARCHAR2(200)	NOT NULL	Legal or trade name
PARTY_TYPE	VARCHAR2(30)	NOT NULL	COUNTERPARTY, ISSUER, BROKER, CUSTODIAN
TAX_ID	VARCHAR2(30)		Tax identifier
COUNTRY	VARCHAR2(3)		ISO country code
SECTOR	VARCHAR2(50)		Economic sector
RATING	VARCHAR2(10)		Credit rating
IS_ACTIVE	NUMBER(1)	NOT NULL	1 = active, 0 = inactive

2.3 INSTRUMENT

Column	Type	Constraint	Description
INSTRUMENT_ID	NUMBER	PK	Unique instrument identifier
TICKER	VARCHAR2(20)		Exchange ticker symbol
INSTRUMENT_NAME	VARCHAR2(200)	NOT NULL	Descriptive name
INSTRUMENT_TYPE	VARCHAR2(30)	NOT NULL	BOND, EQUITY, FUND, DERIVATIVE, DEPOSIT
ASSET_CLASS	VARCHAR2(30)	NOT NULL	FIXED_INCOME, EQUITY, ALTERNATIVES
ISSUER_ID	NUMBER	FK	Reference to PARTY (issuer)
CURRENCY	VARCHAR2(3)	NOT NULL	Denomination currency
MATURITY_DATE	DATE		Maturity date (if applicable)
COUPON_RATE	NUMBER(8, 4)		Coupon rate (%) for fixed income
NOMINAL_VALUE	NUMBER(18, 2)		Face value
IS_ACTIVE	NUMBER(1)	NOT NULL	1 = active, 0 = inactive

3. Sample dataset

The DDL script must include the following seed data. The candidate may add more records but must include at least these to ensure consistent evaluation.

3.1 PARTY (10 records)

ID	Name	Type	Tax ID	Country	Sector	Rating	Active
1	Banco de Chile	COUNTERPARTY	97-004-000-5	CL	Banking	AA	1
2	Tesoreria General	ISSUER	60-805-000-0	CL	Government	AAA	1
3	Falabella S.A.	ISSUER	90-743-000-0	CL	Retail	A+	1
4	BTG Pactual	BROKER	30-306-294-3	BR	Banking	AA-	1
5	Goldman Sachs	COUNTERPARTY	13-5108880	US	Banking	A+	1
6	Ecopetrol S.A.	ISSUER	899999068-1	CO	Energy	BBB	1
7	Corp. Nacional del Cobre	ISSUER	61-704-000-K	CL	Mining	A+	1
8	AFP Habitat	COUNTERPARTY	98-000-100-8	CL	Pension	AA+	1
9	Itau Unibanco	BROKER	60-872-504-4	BR	Banking	AA-	1
10	Inactive Corp	COUNTERPARTY	99-999-999-9	CL	Other	NR	0

3.2 INSTRUMENT (8 records)

ID	Ticker	Name	Type	Asset class	Issuer	CCY	Maturity	Coupon	Active
1	BTP-0230	Bono Tesoreria 2030	BOND	FIXED_INCOME	2	CLP	2030-07-01	3.50	1
2	BCU-0528	BCU 5Y UF 2028	BOND	FIXED_INCOME	2	CLF	2028-05-15	2.80	1
3	FALABELLA	Falabella Equity	EQUITY	EQUITY	3	CLP		0	1
4	ECO-2031	Ecopetrol Bond 2031	BOND	FIXED_INCOME	6	USD	2031-11-20	5.25	1

ID	Ticker	Name	Type	Asset class	Issuer	CC Y	Maturity	Coupon	Active
5	CODELC O29	Codelco Bond 2029	BOND	FIXED_INCOME	7	USD	2029-03-15	4.10	1
6	FONDO-A	Fondo Mutuo Alpha	FUND	ALTERNATIVES	1	CLP		0	1
7	USD-DEP	USD Time Deposit 90d	DEPOSIT	FIXED_INCOME	5	USD	2026-07-15	5.40	1
8	OLD-BOND	Expired Bond 2020	BOND	FIXED_INCOME	3	CLP	2020-01-01	6.00	0

3.3 TRANSACTION (15 records)

ID	Date	Type	Qty	Price	Amount	CC Y	Status	Settl. Date	Port.	Inst.	Cp ty
1	2026-01-10	BUY	50000	98.50	4925000	CLP	SETTLED	2026-01-12	100	1	1
2	2026-01-15	BUY	10000	102.30	1023000	CLF	SETTLED	2026-01-17	100	2	1
3	2026-01-20	BUY	5000	2350	1175000	CLP	SETTLED	2026-01-22	200	3	4
4	2026-02-01	BUY	200000	99.75	1995000	USD	SETTLED	2026-02-03	100	4	5
5	2026-02-10	BUY	300000	100.10	3003000	USD	SETTLED	2026-02-12	100	5	5
6	2026-02-15	SELL	25000	99.20	2480000	CLP	SETTLED	2026-02-17	100	1	8
7	2026-03-01	BUY	100000	10.50	1050000	CLP	APPROVED	2026-03-03	200	6	1
8	2026-03-05	COUPON	50000	3.50	1750	CLP	SETTLED	2026-03-05	100	1	2
9	2026-03-10	BUY	500000	100.00	5000000	USD	PENDING		300	7	5
10	2026-03-15	SELL	2500	2480	6200000	CLP	PENDING		200	3	9
11	2026-03-20	DIVIDEND	5000	120	600000	CLP	SETTLED	2026-03-22	200	3	3
12	2026-04-01	BUY	100000	98.90	9890000	USD	APPROVED	2026-04-03	300	4	4

ID	Date	Type	Qty	Price	Amount	CC Y	Status	Settl. Date	Port.	Inst.	Cpty
13	2026-04-05	MATURITY	100000	100.00	10000000	CLP	SETTLED	2026-04-05	100	8	3
14	2026-04-10	SELL	1500000	100.50	1507500	USD	CANCELLED		100	5	5
15	2026-04-12	BUY	20000	101.20	2024000	CLF	PENDING		100	2	8

4. Metadata configuration

The engine must read its data model structure from configuration. The candidate can choose the storage format (database tables, JSON, YAML) as long as adding a new table or column to the model requires **zero Java code changes**.

4.1 Table and column metadata

The metadata must include at least:

- **Table:** logical name, physical name, default alias, description.
- **Column:** physical name, data type, PK flag, FK reference (target table + column), filterable flag, selectable flag.

4.2 Relationship metadata (JOINS)

Relationships must be defined in metadata so the engine can automatically resolve JOINS.

Source table	Source column	Target table	Target column	Join type	Relation alias
TRANSACTION	INSTRUMENT_ID	INSTRUMENT	INSTRUMENT_ID	LEFT JOIN	instrument
TRANSACTION	COUNTERPARTY_ID	PARTY	PARTY_ID	LEFT JOIN	counterparty
INSTRUMENT	ISSUER_ID	PARTY	PARTY_ID	LEFT JOIN	issuer

NOTE: When a query involves columns from multiple tables, the engine must resolve the JOIN path automatically. If PARTY is referenced, the engine should determine whether to join via COUNTERPARTY_ID or ISSUER_ID based on the relationship alias or an explicit hint in the request.

5. REST API specification

5.1 POST /api/query/build

Receives a high-level query specification and returns the generated SQL. The request uses an **abstract, domain-oriented format** — not raw SQL fragments. The engine is responsible for translating this into valid SQL.

Request body

```
{ "select": [ { "entity": "transaction", "field": "txnDate" }, { "entity": "transaction",
"field": "amount" }, { "entity": "transaction", "field": "currency" }, { "entity":
"instrument", "field": "ticker" }, { "entity": "instrument", "field": "instrumentName" },
{ "entity": "counterparty", "field": "partyName", "alias": "counterpartyName" } ],
"filters": { "operator": "AND", "conditions": [ { "entity": "transaction", "field":
"status", "comparator": "equals", "value": "SETTLED" }, { "entity": "transaction",
"field": "amount", "comparator": "greaterThan", "value": 1000000 }, { "operator": "OR",
"conditions": [ { "entity": "instrument", "field": "assetClass", "comparator": "in",
"value": ["FIXED_INCOME", "EQUITY"] }, { "entity": "counterparty", "field": "country",
"comparator": "equals", "value": "CL" } ] } ] }, "sorting": [ { "entity": "transaction",
"field": "txnDate", "direction": "desc" } ], "maxResults": 500 }
```

Key design points of the request format:

- **Entity + field** instead of TABLE.COLUMN: the consumer uses logical names (transaction, counterparty, instrument), not physical table names.
- **Relationship aliases**: "counterparty" maps to PARTY via COUNTERPARTY_ID; "issuer" maps to PARTY via ISSUER_ID. The engine resolves the correct JOIN path.
- **Nested filter groups**: filters use a recursive tree structure with operator (AND/OR) and conditions. This allows complex boolean logic without flattening to a simple list.
- **Semantic comparators**: equals, notEquals, greaterThan, lessThan, greaterOrEqual, lessOrEqual, in, notIn, between, like, isNull, isNotNull. The engine maps these to SQL operators.
- **camelCase field names**: the engine maps them to physical SNAKE_CASE column names using metadata.

Response body (200 OK)

```
{ "sql": "SELECT t.TXN_DATE, t.AMOUNT, t.CURRENCY, ...", "parameters": { "p1": "SETTLED",
"p2": 1000000, "p3": ["FIXED_INCOME", "EQUITY"], "p4": "CL" }, "resolvedTables":
["TRANSACTION", "INSTRUMENT", "PARTY"], "resolvedJoins": [ "LEFT JOIN INSTRUMENT i ON
t.INSTRUMENT_ID = i.INSTRUMENT_ID", "LEFT JOIN PARTY p ON t.COUNTERPARTY_ID = p.PARTY_ID"
], "metadata": { "columnCount": 6, "filterCount": 4, "generatedAt":
"2026-04-14T15:30:00Z" } }
```

NOTE: The parameters map contains the bind variables. The SQL must use named parameters (:p1, :p2) or positional placeholders — never inline values. This is critical for SQL injection prevention.

5.2 GET /api/metadata/entities

Returns the list of available entities with their fields, types, and relationships.

Response body (200 OK)


```
[ { "entity": "transaction", "physicalTable": "TRANSACTION", "alias": "t", "fields": [ {
  "name": "txnId", "physicalName": "TXN_ID", "type": "number", "primaryKey": true,
  "filterable": true, "selectable": true }, { "name": "txnDate", "physicalName":
  "TXN_DATE", "type": "date", "filterable": true, "selectable": true }, ... ], "relations":
  [ { "alias": "instrument", "targetEntity": "instrument", "joinType": "LEFT",
    "sourceField": "instrumentId", "targetField": "instrumentId" }, ... ] }, ... ]
```

5.3 GET /api/metadata/comparators

Returns the valid comparators grouped by data type.

Data type	Valid comparators
string	equals, notEquals, like, in, notIn, isNull, isNotNull
number	equals, notEquals, greaterThan, lessThan, greaterOrEqual, lessOrEqual, between, in, isNull, isNotNull
date	equals, notEquals, greaterThan, lessThan, greaterOrEqual, lessOrEqual, between, isNull, isNotNull
timestamp	equals, notEquals, greaterThan, lessThan, greaterOrEqual, lessOrEqual, isNull, isNotNull

5.4 POST /api/query/validate

Receives the same body as /api/query/build but only validates without generating SQL. Returns structured errors.

Response body (400 Bad Request)

```
{ "valid": false, "errors": [ { "entity": "transaction", "field": "status", "comparator":
  "greaterThan", "message": "Comparator greaterThan is not valid for string fields" }, {
  "entity": "unknownEntity", "message": "Entity unknownEntity does not exist in the model"
} ] }
```

6. Technical requirements

6.1 Mandatory

- **Java 8+ and Spring Boot 2.7+:** the project must compile and run with Maven or Gradle.
- **Database:** embedded H2 in Oracle compatibility mode. The DDL script must create the data model tables, metadata tables, and load the seed data from Section 3.
- **Metadata-driven:** the model definition (entities, fields, relationships, comparators) must come from configuration, not code.
- **Automatic JOIN resolution:** the engine must determine which JOINS are needed based on the entities and fields requested.
- **Logical-to-physical mapping:** the engine must translate camelCase field names to SNAKE_CASE columns and entity aliases to physical table names using metadata.
- **Nested filter groups:** the filter tree must support arbitrary nesting of AND/OR groups.
- **Input validation:** reject queries with unknown entities/fields, invalid comparators for the data type, and malformed values. Return structured error responses.
- **SQL injection prevention:** filter values must be handled as bind parameters, never concatenated into the SQL string.
- **Testing:** unit tests for the SQL generation engine (filter parsing, JOIN resolution, name mapping, validation) and at least 2 integration tests for the API endpoints.
- **Documentation:** README.md with setup instructions, design decisions, patterns used and their rationale.

6.2 Mandatory use of AI

The candidate **must** use Claude Code (or another AI-assisted development tool) during implementation. Include a section in the README titled "**AI usage in development**" covering:

- Which AI tool(s) you used.
- Which specific tasks the AI helped with (design, code, tests, debugging, documentation).
- Cases where you had to correct, discard, or redo what the AI generated, and why.
- What you learned about effective AI usage in development during this exercise.

6.3 Bonus (not required)

- **Function support in filters:** e.g. `upper(partyName) like 'BANCO%'`.
- **Aggregations:** support GROUP BY with aggregate functions (sum, count, avg) in selected fields.
- **Pagination:** generate SQL with OFFSET/FETCH or Oracle-style ROWNUM.
- **Metadata caching:** load metadata into memory on startup with a reload endpoint.

- **Query execution endpoint:** POST /api/query/execute that generates and runs the SQL against H2, returning actual results.
- **Swagger/OpenAPI:** auto-generated API documentation.

7. Evaluation criteria

Evaluation follows this priority order:

#	Criterion	Weight	What we evaluate
1	Design & patterns	30%	Separation of concerns, patterns applied with judgment (Strategy, Builder, Composite, Interpreter, etc.), abstractions that allow extension without modification, domain modeling.
2	Code quality	25%	Readability, naming, SOLID principles applied pragmatically, error handling, consistency across the codebase.
3	Metadata-driven engine	15%	Configuration is truly external. Adding a new entity/field requires zero Java code changes.
4	Testing	15%	Edge case coverage (empty filters, unknown entities, type mismatches, nested groups), maintainable tests, unit + integration mix.
5	AI usage	10%	Honest and reflective README section. Code feels coherent, not a patchwork of copied fragments.
6	Documentation	5%	Clear README. Decisions explained, not just execution mechanics.

IMPORTANT: We are not looking for over-engineering. A Builder that simplifies SQL construction is good; a hierarchy of 8 abstract classes for 3 entities is excessive. We value the judgment to choose the right level of abstraction for the problem at hand.

8. Delivery

- **Git repository** (GitHub or GitLab) with access granted to evaluators.
- **Commit history** that reflects the development process. We do not want a single monolithic commit.
- **README.md** including: how to run the project, how to run the tests, design decisions and patterns used, AI usage section.
- **DDL script** included: must create the schema, metadata tables, and load the seed data.
- **The project must compile and run** with a single command (`mvn spring-boot:run` or `./gradlew bootRun`).

Deadline

You have **3 calendar days** from receipt of this document. The estimated development effort is 6–8 hours. If you anticipate any issues with the deadline, communicate in advance.

If you have questions about the specification, email your recruiting contact. We prefer that you ask rather than assume — that is also part of what we evaluate.